Proposed change [A93], the loop figure and the text around it

13 November 2026, version 3 (blocking cost and crowding feature, Part 1 and 2.3 to 2.5)

How to read this

Part 1 says why the figure changes. Part 2 shows the section "How the two layers close the loop" as it would read. Green is new text, ~~struck~~ text goes, black text stays. Part 3 has the questions. Only the RQ number is changed in the thesis so far, and the RQ numbers.

Part 1. Why the figure changes

What the reader already has

- **The loop in words.** The Introduction already explains both directions, and that demand storage runs one way while closing the loop runs both ways. This section is the second time the reader meets the loop.
- **The two clocks.** The environment step figure of the Implementation chapter (`methodloop.pdf`) shows every step against the storage epoch.
- **The tea example** of Figure `storageexample`, with two aisles, a workstation and new tea placed by demand storage.
- **The agents' own crowding feature** δ of Figure `congestion`. It is easy to confuse with what storage reads.

What the reader is missing

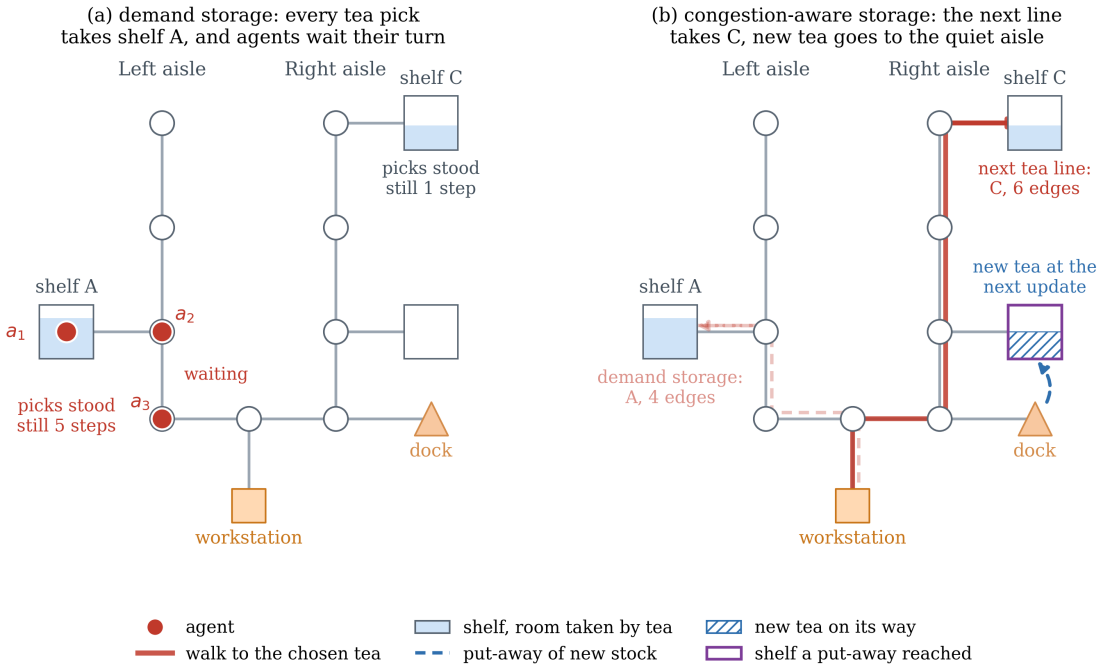
The storage section ends about 500 lines earlier, so the reader has forgotten its details. "Standing still at a shelf" has only been said in words. It is the one new idea of the section, and no figure in the thesis shows it.

What the old figure did

It drew boxes and arrows with symbols (x_t^{in}, τ_j, $\hat{\mu}_t$). The idea itself is already in the Introduction and in the section's own words. The figure repeated the abstract part and left out the concrete part. It also did not show demand storage, so the caption had to.

The new figure

One figure, two panels. It carries the tea example of Figure `storageexample` one storage period further, drawn in the same way. Shelves are slot boxes hanging off the aisle, and agents stand only on vertices, as in Figure `storageupdate`. (a) Demand storage sends every tea pick to shelf A, the nearest. Agent a_1 picks there while a_2 and a_3 wait their turn on the aisle. (b) Congestion-aware storage reads that. The next line takes C, 6 edges from the workstation against 4 for A, because $6 + 1 < 4 + 5$. At the next update the new tea goes to the empty shelf of the quiet aisle. The numbers are those of the pickup example in "Orders and tasks". The corner of Figure `taskroute` was tried first, but every trip to W_1 passes the same two vertices there, so a second tea shelf could not be quiet.



One name per idea

"Crowding" meant two things. The share of an agent's view taken by other agents, $\delta_i(t)$, and the cost K of the storage rule. Since [A93], K is built from the time picks stood still, so it charges blocking, not crowding. The thesis already says crowding without blocking costs nothing. K also had a second name, "congestion term", and $\delta_i(t)$ had two, "congestion feature" and "crowding number". Now each idea has one name.

idea	name
$\delta_i(t)$, what an agent sees around it	crowding feature
$\hat{\omega}_t(v)$, steps picks stood still at a shelf	standing still
K, built from $\hat{\omega}_t$	blocking cost
the rule and its weight β	congestion-aware storage, congestion weight

"Blocking cost" matches the blocked part of eq. split, which $\hat{\omega}_t$ averages, and the blocked time RQ2 asks about. "Waiting cost" was not used, because waiting already names a task waiting for an agent. The labels `eq:congestion` and `fig:congestion` stay, since the reader does not see them.

Why storage does not read the crowding feature

A crowd that keeps moving costs no time, and a busy aisle that flows is often the best place for popular stock. Standing still counts only time that is lost. It is in steps, the unit of a trip, so $\beta = 1$ has a plain meaning, while δ is a share from 0 to 1 and would need a tuned weight. It is booked on the shelf a pick came from, so it also catches a queue at the workstation. And it is what RQ2 measures. A storage rule on δ itself was never run. The nearest test, passes per edge, did no better than demand storage with Token Passing on 10 seeds.

Part 2. The text

Problem Formulation, "How the two layers close the loop"

The two layers of the warehouse run at different speeds. Routing acts on every timestep. Only every Δ timesteps, at the storage epochs, does new stock arrive and the storage rule place it, through eq. storageupdate.

The two layers influence each other in both directions, and that is what closes the loop. Forward, storage shapes routing. Placing stock changes which vertices can be pickups under the stock condition, and so changes where the agents have to go.

Backward, routing shapes storage. When many picks go to one shelf, the agents wait their turn there. How long ~~agents~~ picks stand still is measured at each shelf in $\hat{\omega}_t$. At the next storage update the rule uses it to decide where new stock goes, and every new order line uses it to choose the shelf it picks from. This is not the crowding an agent sees around itself, the crowding feature of Figure `congestion`, which only the controller reads. Storage reads standing still instead, because a crowd that keeps moving costs no time. Figure `loop` carries the tea example of Figure `storageexample` one storage period further, with the numbers of the pickup example in Section `pf:tasks`.

Figure: shelf_loop.png, at 0.9 of the line width, label fig:loop kept. Caption: The tea example of Figure `storageexample` a period later. Picks wait at A, so the next line and the new tea go to the quiet aisle. ~~The two-timescale loop under fixed (a) and congestion-aware storage (b). Demand storage reads only $\hat{\rho}_t$, so standing still never reaches it.~~

With fixed storage only the forward path exists. Demand storage places stock by the orders, and the orders come from customers, not from how the agents move. So with demand storage the coupling still runs one way, as a gain that depends on the controller. The backward path is what congestion-aware storage adds, because only that rule reads the time agents stand still. ~~Demand storage follows the orders alone. With demand storage the coupling therefore shows only one way, as a gain that depends on the controller.~~ The loop is closed only in ~~RQ3~~ RQ2, which tests it at the main load and at a higher load where the aisles are crowded. ~~Figure `loop` shows this. The fixed-storage baseline comes first, and the congestion-aware model is the same diagram with the backward path added.~~

Separating the two clocks has one consequence that constrains the study. *The paragraph on* Δ *stays as it is.*

Implementation, "Where arriving stock goes"

... The gap between the last two shows what reading the time agents stand still adds to demand alone, which is the storage half of ~~RQ3~~ RQ2.

2.3 Problem Formulation, "What the rule decides"

The ~~crowding~~ blocking cost is this thesis's own, because the classic problem has no traffic. It adds up, over the same units, how often their SKU is ordered times how long picks stand still at their shelf. ...

2.4 The other places of K

Background, "Congestion and emergence". There are two ways to respond that matter here. The first gives every controller an explicit crowding signal instead of hoping the fleet corrects itself. The ~~congestion~~ crowding feature does this for routing, and the ~~congestion term~~ blocking cost K does it for storage, through the time picks stood still. Routing can then react to crowding before it costs time, and storage to the time it has already cost. ~~does it for storage. Both layers can then react to crowding before it costs time.~~

Problem Formulation, "Crowding". ... The storage rule guards against it through the ~~congestion term~~ blocking cost K ...

Notation table. D, K: Its two terms, the slotting cost and the ~~crowding~~ blocking cost. β: How much the ~~crowding~~ blocking cost K, steps standing still, counts against the slotting cost D.

2.5 The crowding feature $\delta_i(t)$

The same change, ~~congestion feature~~ or ~~crowding number~~ to crowding feature, in eight places. Problem Formulation, "How crowded the neighbourhood is" ("A second number, the crowding feature $\delta_i(t)$, ...") and the caption of Figure `congestion` ("The crowding feature $\delta_i(t)$ is measured the same way for every agent..."). Implementation, two places ("The crowding feature $\delta_i(t)$ is added once for the agent", "so the crowding feature means the same under every controller"). Background, one place (2.4). Appendix C ("The crowding feature,"), Appendix K ("The crowding feature $\delta_i(t)$ is summed over agents") and the notation table (d_{obs}, "shared by every architecture's crowding feature"). The reward's "congestion term" charges crowding through δ and keeps its name, because it is a different term. The [A93] entry in `GAPS.md` records both renames.

Part 3. Questions, answered 13 November

1. **Names.** Shelves are slot boxes named shelf A and shelf C, as in Table I. Agents are a_1, a_2, a_3. Every agent stands on an aisle vertex or the shelf it picks from.
2. **The faint arrow in (b).** Kept.
3. **The sentence on δ.** Kept.
4. **The pickup example.** No pointer ahead from "Orders and tasks". The loop section points back to it instead (Part 2, green).
5. **Other old RQ numbers.** Done in the working copy, old RQ1 and RQ2 are now RQ1, old RQ3 to RQ5 are now RQ2 to RQ4. Background (2), Problem Formulation (5), Implementation (1), Appendix E (3) and the Experiments chapter (17, the old RQ1 and RQ2 sentences joined into one). One to check. The Background says crowding signals and messages are tested "through RQ2". Messages are RQ4, so it may be "RQ2 and RQ4".

6. **The old figure files.** `loop.py`, `loop.tex` and `loop.pdf` are removed from `scripts/images`, and `figure.py` now draws `shelf_loop.png`. `loop.pdf` in `thesis/Images` goes when the new figure is applied.

Still open. Approve the figure and the text of Part 2, then they go into the thesis.